Python Flask
Aplikacje Webowe

Podręcznik kursowy - kod

Mobilo © 2021


RAFAŁ MOBILO24.EU © 2020

Spis treści

Wstęp - Dlaczego Flask?

Tutaj znajdziesz fragmenty kodu, które będzie ewentualnie łatwiej kopiować i wklejać do swoich programów w razie potrzeby.

Środowisko wirtualne, instalacja Flask, pierwszy program

Propozycja rozwiązania (tylko kod aplikacji)

```
from flask import Flask
from datetime import datetime

app = Flask(__name__)

@app.route('/')
def index():
    time_now =  datetime.now().strftime('%H:%M:%S')
    return '<h1>Hello World: {}</h1>'.format(time_now)

@app.route('/links')
def links():
    body = '''<a href="http://www.google.com" target="_blank">Google</a> <br />
              <a href="http://www.bing.com" target="_blank">Default search engine to
find Google</a>'''
    return  body

if __name__ == '__main__':
    app.run()
```


RAFAŁ MOBILO24.EU © 2020

Terminal, zmienne środowiskowe i typowe problemy

Propozycja rozwiązania

```
set FLASK_APP=time_app.py
set FLASK_DEBUG=1

Na Linux

export FLASK_APP=time_app.py
export FLASK_DEBUG=1
```

Dynamiczny routing

Propozycja rozwiązania

```
@app.route('/cook/<string:receipt>/<int:step>')
def cook(receipt, step):
    body = f'''<H1>In the receipt {receipt} you are on step {step}</H1>'''
    return  body
```


RAFAŁ MOBILO24.EU © 2020

Przekazywanie parametrów przez Query String

Propozycja rozwiązania

```
@app.route('/cook/<string:receipt>/<int:step>')
def cook(receipt, step):

    print(request.query_string)

    print('----')

    print(request.args)

    font_size='100%'
    if 'font-size' in request.args:
        font_size = request.args['font-size']

    body = f'''<H1 style="font-size: { font_size }">In the receipt {receipt} you are
on step {step}</H1>'''
    return  body
```

Przyjmowanie danych z formularzy

Propozycja rozwiązania

```
from flask import Flask, request

# …

@app.route('/rate_receipt')
def rate_receipt():

    body = '''
        <form id="rating" action="/rate_receipt_save" method="POST">
            <label for=note>What is your note for the receipt?</label><br>
            <select id="nore" name="note">
                <option value="5">It is great!</option>
                <option value="4">It is very good</option>
                <option value="3" selected>It is just good</option>
                <option value="2">It was poor</option>
                <option value="1">It was horrible!</option>
            </select><br>

            <label for=comment>Write down your comments:</label><br>
            <textarea id="comment" name="comment" rows="3" cols="50">
            </textarea><br>

            <label for="decision">Would you cook it for your family?</label><br>
            <input type="checkbox" id="decision" name="decision"><br>

            <input type="submit" value="Share my feedback">
        </form>
    '''
    return body


@app.route('/rate_receipt_save', methods=['POST'])
def rate_receipt_save():

    note = 3
    if 'note' in request.form:
        note = request.form['note']
    comment=''
    if 'comment' in request.form:
        comment = request.form['comment']
    decision = False
    if 'decision' in request.form:
        decision = True

    message = f'''Your rating was: {note}<br>
                  Your comment was: {comment}<br>
                  Your decision was {decision}
    '''
    return message
```


RAFAŁ MOBILO24.EU © 2020

Obsługa GET i POST w jednej funkcji. Obiekt request

Propozycja rozwiązania

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/rate_receipt', methods=['GET', 'POST'])
def rate_receipt():

    if request.method == 'GET':
        body = '''
            <form id="rating" action="/rate_receipt" method="POST">
                <label for=note>What is your note for the receipt?</label><br>
                <select id="nore" name="note">
                    <option value="5">It is great!</option>
                    <option value="4">It is very good</option>
                    <option value="3" selected>It is just good</option>
                    <option value="2">It was poor</option>
                    <option value="1">It was horrible!</option>
                </select><br>

                <label for=comment>Write down your comments:</label><br>
                <textarea id="comment" name="comment" rows="3" cols="50">
                </textarea><br>

                <label for="decision">Would you cook it for your family?</label><br>
                <input type="checkbox" id="decision" name="decision"><br>

                <input type="submit" value="Share my feedback">
            </form>
        '''
        return body
    else:
        note = 3
        if 'note' in request.form:
            note = request.form['note']
        comment=''
        if 'comment' in request.form:
            comment = request.form['comment']
        decision = False
        if 'decision' in request.form:
            decision = True

        message = f'''Your rating was: {note}<br>
                    Your comment was: {comment}<br>
                    Your decision was {decision}
        '''
        return message


# …
```

Funkcja url_for – Twój przyjaciel w budowaniu linków

Propozycja rozwiązania

```
from flask import Flask, request, url_for

app = Flask(__name__)

@app.route('/rate_receipt', methods=['GET', 'POST'])
def rate_receipt():

    if request.method == 'GET':
        body = f'''
            <form id="rating" action="{ url_for('rate_receipt') }" method="POST">

#... etc
```


RAFAŁ MOBILO24.EU © 2020

Funkcja redirect, czyli “zapraszamy do kasy obok”

Propozycja rozwiązania

```
from flask import Flask, url_for, redirect

app = Flask(__name__)

@app.route('/not_implemented/<message>')
def not_implemented(message):
    return '<h1 style="color:red">{}</h1>'.format(message)

@app.route('/new_receipt')
def new_receipt():
    return redirect(url_for('not_implemented', message="Function new_receipt is not
ready yet"))

@app.route('/delete_receipt/<name>')
def delete_receipt(name):
    return redirect(url_for('not_implemented', message="Function new_receipt is not
ready yet"))
```

Statyczne elementy projektu

Propozycja rozwiązania

```
from flask import Flask, url_for, redirect
import os

app = Flask(__name__)

@app.route('/download/<file>')
def download(file):

    subpath = 'download/'+file
    local_file_path = os.path.join(app.static_folder, subpath)

    if(os.path.isfile(local_file_path)):
        print('file found')
        return redirect(url_for('static', filename=subpath))
    else:
        print('sorry file not found')
        return 'File not found!'
```


RAFAŁ MOBILO24.EU © 2020

Definiowanie środowiska Flask

Propozycja rozwiązania

W pliku activate dodaj: SET FLASK_ENV=development lub EXPORT FLASK_ENV an Linuxie

```
flask routes
```

Propozycja rozwiązania

```
#  templates/notification.html

<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>
        <form id="notification_form" method="POST" action="{{ url_for('notification')
}}">
            <label for="room_number">Room number</label>
            <input type="text" name="room_number" id="room_number"><br>
            <label for="guest_name">Guest name</label>
            <input type="text" name="guest_name" id="guest_name"><br>
            <label for="notification_text">Notification</label>
            <textarea rows="4" cols="50" name="notification_text"
id="notification_text"></textarea><br>
            <input type="submit" value="Send">
        </form>
    </body>
</html>

# templates/notification_content.html

<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>
        <h1>Notification content</h1>
        <ul>
            <li style="font-weight: bold;">Room number</li>
            <li>{{ room_number }}</li>
            <li style="font-weight: bold;">Guest name</li>
            <li>{{ guest_name }}</li>
            <li style="font-weight: bold;">Notification</li>
            <li>{{ notification_text }}</li>
        </ul>
    </body>
</html>

#  app.py

from flask import Flask, url_for, request, render_template

app = Flask(__name__)

@app.route('/notification', methods=['GET', 'POST'])
def notification():

    if request.method == 'GET':
        return render_template('notification.html')
    else:
        room_number = request.form['room_number'] if 'room_number' in request.form
else ''
        guest_name = request.form['guest_name'] if 'guest_name' in request.form else
''
        notification_text =  request.form['notification_text'] if 'notification_text'
in request.form else ''

        return render_template('notification_content.html',
                room_number=room_number, guest_name=guest_name,
notification_text=notification_text)
```


RAFAŁ MOBILO24.EU © 2020

Warunkowe wyświetlanie części szablonu Jinja

Propozycja rozwiązania

```
#  template notification.html (only added part)

            <label for="priority">Priority</label>
            <select name="priority" id="priority">
                <option value="high">HIGH PRIORITY</option>
                <option value="medium">MEDIUM</option>
                <option value="normal" selected>NOT URGENT</option>
            </select><br>

# ----- getting new field value from the request (only modified part)

@app.route('/notification', methods=['GET', 'POST'])
def notification():

    if request.method == 'GET':
        return render_template('notification.html')
    else:
        room_number = request.form['room_number'] if 'room_number' in request.form
else ''
        guest_name = request.form['guest_name'] if 'guest_name' in request.form else
''
        notification_text =  request.form['notification_text'] if 'notification_text'
in request.form else ''
        priority = request.form['priority'] if 'priority' in request.form else
'normal'

        return render_template('notification_content.html',
                room_number=room_number, guest_name=guest_name,
                notification_text=notification_text, priority=priority)

#   template notification_confirmation.html. Only added part

    <body>
        {% if priority=='high' %}
            <h1>Critical notification content</h1>
        {% elif priority=='medium' %}
            <h1>Important notification content</h1>
        {% else %}
             <h1>Notification content</h1>
        {% endif %}
```


RAFAŁ MOBILO24.EU © 2020

Pętle w szablonach Jinja

Propozycja rozwiązania

app.py:

```
from flask import Flask, url_for, request, redirect, render_template
import os

app = Flask(__name__)

class PriorityType:
    def __init__(self, code, description, selected):
        self.code = code
        self.description = description
        self.selected = selected

class NotificationPriorities:
    def __init__(self):
        self.list_of_priorities = []

    def load_priorities(self):
        self.list_of_priorities.append(PriorityType('high','HIGH PRIORITY', False))
        self.list_of_priorities.append(PriorityType('medium','MEDIUM', False))
        self.list_of_priorities.append(PriorityType('normal', 'NOT URGENT', True))
        self.list_of_priorities.append(PriorityType('low', 'REMARK', False))

    def get_priority_by_code(self, code):
        for p in self.list_of_priorities:
            if p.code == code:
                return p
        return PriorityType('normal', 'NOT URGENT', True)

@app.route('/notification', methods=['GET', 'POST'])
def notification():

    notification_priorities = NotificationPriorities()
    notification_priorities.load_priorities()

    if request.method == 'GET':
        return render_template('notification.html',
                               list_of_priorities=notification_priorities.list_of_priorities)
    else:
        room_number = request.form['room_number'] if 'room_number' in request.form else ''
        guest_name = request.form['guest_name'] if 'guest_name' in request.form else ''
        notification_text =  request.form['notification_text'] if 'notification_text' in
request.form else ''
        priority = request.form['priority'] if 'priority' in request.form else 'normal'

        priority_type = notification_priorities.get_priority_by_code(priority)
        print('found', priority_type.code)

        return render_template('notification_content.html',
                room_number=room_number, guest_name=guest_name,
                notification_text=notification_text, priority_type=priority_type)
```

notification.html:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>
        <form id="notification_form" method="POST" action="{{ url_for('notification') }}">
            <label for="room_number">Room number</label>
            <input type="text" name="room_number" id="room_number"><br>
            <label for="guest_name">Guest name</label>
            <input type="text" name="guest_name" id="guest_name"><br>
            <label for="notification_text">Notification</label>
            <textarea rows="4" cols="50" name="notification_text"></textarea><br>
            <label for="priority">Priority</label>
            <select name="priority" id="priority">
                {% for priority in list_of_priorities %}
                    <option value="{{ priority.code }}" {{ 'selected' if priority.selected}}>{{
priority.description }}</option>
                {% endfor %}
            </select><br>
            <input type="submit" value="Send">
        </form>
    </body>
</html>
```

notification_confirmation.html:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>
        {% if priority_type.code=='high' %}
            <h1>Critical notification content</h1>
        {% elif priority_type.code=='medium' %}
            <h1>Important notification content</h1>
        {% else %}
             <h1>Notification content</h1>
        {% endif %}
        <ul>
            <li style="font-weight: bold;">Room number</li>
            <li>{{ room_number }}</li>
            <li style="font-weight: bold;">Guest name</li>
            <li>{{ guest_name }}</li>
            <li style="font-weight: bold;">Notification</li>
            <li>{{ notification_text }}</li>
        </ul>
    </body>
</html>
```


RAFAŁ MOBILO24.EU © 2020

Funkcja flash() – czyli krótkie info o statusie

Propozycja rozwiązania

```
# in the app.py

from flask import flash
from datetime import datetime

# …

app = Flask(__name__)
app.config['SECRET_KEY'] = '123GoniszTy!'

# …

# somewhere in the function notification()

        flash('Notification has been sent')

        the_hour = datetime.now().hour
        raise_priority = (the_hour >= 20 or the_hour < 6) and priority == 'medium'

        if raise_priority:
            priority = 'high'
            flash('Rising priority from medium to high')



# in the nothification_content.html

        {% for message in get_flashed_messages() %}
            <div>{{ message }}</dev>
        {% endfor %}
```

Makra Jinja

Propozycja rozwiązania

```
# file macros.html

{% macro show_flash() %}
    <ul>
    {% for message in get_flashed_messages() %}
        <li>{{ message }}</li>
    {% endfor %}
    </ul>
{% endmacro %}



# file notification_content.html

{% from "macros.html" import show_flash %}
<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>

        {{ show_flash() }}

        {% if priority_type.code=='high' %}
            <h1>Critical notification content</h1>
        {% elif priority_type.code=='medium' %}
            <h1>Important notification content</h1>
        {% else %}
             <h1>Notification content</h1>
        {% endif %}
        <ul>
            <li style="font-weight: bold;">Room number</li>
            <li>{{ room_number }}</li>
            <li style="font-weight: bold;">Guest name</li>
            <li>{{ guest_name }}</li>
            <li style="font-weight: bold;">Notification</li>
            <li>{{ notification_text }}</li>
        </ul>
    </body>
</html>
```


RAFAŁ MOBILO24.EU © 2020

Propozycja rozwiązania

```
# base.html :

{% from "macros.html" import show_flash %}
<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>
        {{ show_flash() }}

        {% block page_body %}
        {% endblock %}
    </body>
</html>



# notification.html :

{% extends "base.html" %}

{% block page_body %}
    <form id="notification_form" method="POST" action="{{ url_for('notification') }}">
        <label for="room_number">Room number</label>
        <input type="text" name="room_number" id="room_number"><br>
        <label for="guest_name">Guest name</label>
        <input type="text" name="guest_name" id="guest_name"><br>
        <label for="notification_text">Notification</label>
        <textarea rows="4" cols="50" name="notification_text"></textarea><br>
        <label for="priority">Priority</label>
        <select name="priority" id="priority">
            {% for priority in list_of_priorities %}
                <option value="{{ priority.code }}" {{ 'selected' if
priority.selected}}>{{ priority.description }}</option>
            {% endfor %}
        </select><br>
        <input type="submit" value="Send">
    </form>
{% endblock %}


# notification_content.html

{% extends "base.html" %}

{% block page_body %}
    {% if priority_type.code=='high' %}
        <h1>Critical notification content</h1>
    {% elif priority_type.code=='medium' %}
        <h1>Important notification content</h1>
    {% else %}
            <h1>Notification content</h1>
    {% endif %}
    <ul>
        <li style="font-weight: bold;">Room number</li>
        <li>{{ room_number }}</li>
        <li style="font-weight: bold;">Guest name</li>
        <li>{{ guest_name }}</li>
        <li style="font-weight: bold;">Notification</li>
        <li>{{ notification_text }}</li>
    </ul>
{% endblock %}
```

Dołączanie szablonów Jinja - include

Propozycja rozwiązania

```
# file footer.html

<div style="text-align: center; border-style: dotted; font-weight: bold;">
    Py-Hotel
</div>



# file base.html


{% from "macros.html" import show_flash %}
<!DOCTYPE html>
<html>
    <head>
        <title>Hotel 101</title>
    </head>
    <body>
        {{ show_flash() }}

        {% block page_body %}
        {% endblock %}

        {% include 'footer.html' %}
    </body>
</html>
```


RAFAŁ MOBILO24.EU © 2020

Flask Bootstrap

Propozycja rozwiązania

```
# Przygotowanie środowiska:

Python -m venv venv
.\venv\scripts\activate.bat
Pip install flask
Pip install flask-bootstrap
SET FLASK_ENV=development




# Kod aplikacji

from flask import Flask, render_template
from flask_bootstrap import Bootstrap

app = Flask(__name__)
bootstrap = Bootstrap(app)

@app.route('/')
def index():
    return render_template('index.html')




# Kod szablonu index.html

{% extends "bootstrap/base.html" %}

{%block content %}
   <h1>Example heading <span class="badge bg-secondary">New</span></h1>
   <h2>Example heading <span class="badge bg-secondary">New</span></h2>
   <h3>Example heading <span class="badge bg-secondary">New</span></h3>
   <h4>Example heading <span class="badge bg-secondary">New</span></h4>
   <h5>Example heading <span class="badge bg-secondary">New</span></h5>
   <h6>Example heading <span class="badge bg-secondary">New</span></h6>
{% endblock %}
```

Korzystanie z Bootstrap bez Flask Bootstrap

Propozycja rozwiązania

```
# File base.html

<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Bootstrap CSS -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-
beta2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
BmbxuPwQa2lc/FVzBcNJ7UAyJxM6wuqIj61tLrc4wSX0szH/Ev+nYRRuWlolflfl"
crossorigin="anonymous">
    <title>{% block title %} {% endblock %}</title>
  </head>
  <body>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-
beta2/dist/js/bootstrap.bundle.min.js" integrity="sha384-
b5kHyXgcpbZJO/tY9Ul7kGkf1S0CWuKcCD38l8YkeH8z8QjE0GmW1gYU5S9FOnJ0"
crossorigin="anonymous"></script>
  </body>
</html>


# file index.html

{% extends "base.html" %}

{% block title %} Demo page for bootstrap in flask  {% endblock %}

{%block content %}

<!-- Button trigger modal -->
<button type="button" class="btn btn-primary" data-bs-toggle="modal" data-bs-
target="#staticBackdrop">
    Launch static backdrop modal
  </button>

  <!-- Modal -->
  <div class="modal fade" id="staticBackdrop" data-bs-backdrop="static" data-bs-
keyboard="false" tabindex="-1" aria-labelledby="staticBackdropLabel" aria-
hidden="true">
    <div class="modal-dialog">
      <div class="modal-content">
        <div class="modal-header">
          <h5 class="modal-title" id="staticBackdropLabel">Modal title</h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal" aria-
label="Close"></button>
        </div>
        <div class="modal-body">
          ...
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Close</button>
          <button type="button" class="btn btn-primary">Understood</button>
        </div>
      </div>
    </div>
  </div>

{% endblock %}
```


RAFAŁ MOBILO24.EU © 2020

Połącznie kodu z szablonami

Propozycja rozwiązania

```
# menu.html

<ul class="nav nav-pills">
    <li class="nav-item">
      <a class="nav-link active" aria-current="page" href="{{ url_for('index') }}">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" href="#">Notifications</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" href="{{ url_for('notification') }}">New notification</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" href="{{ url_for('about') }}" tabindex="-1">About this app</a>
    </li>
  </ul>

# base.html
…
    <title>{% block title %} {% endblock %}</title>
…
    {% include 'menu.html' %}

    {% block content %}  {% endblock %}
…

# index.html
{% extends "base.html" %}

{% block title %} Hotel notifications {% endblock %}

{%block content %}
<div id="carouselExampleIndicators" class="carousel slide" data-bs-ride="carousel">
  <div class="carousel-indicators">
    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="0"
class="active" aria-current="true" aria-label="Slide 1"></button>
    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="1" aria-
label="Slide 2"></button>
  </div>
  <div class="carousel-inner">
    <div class="carousel-item active">
      <div class="d-flex justify-content-center">
        <img src="{{ url_for('static', filename='carousel/1.jpg') }}" class="d-block" alt="..."
style="width: 90%;">
      </div>
    </div>
    <div class="carousel-item">
      <div class="d-flex justify-content-center">
        <img src="{{ url_for('static', filename='carousel/2.jpg') }}" class="d-block" alt="..."
style="width: 90%;">
      </div>
    </div>
  </div>
  <button class="carousel-control-prev" type="button" data-bs-target="#carouselExampleIndicators"
data-bs-slide="prev">
    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Previous</span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-target="#carouselExampleIndicators"
data-bs-slide="next">
    <span class="carousel-control-next-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Next</span>
  </button>
</div>
{% endblock %}

# app.py

from flask import Flask, url_for, request, redirect, render_template, flash
from datetime import datetime
import os

app = Flask(__name__)
```

```
app.config['SECRET_KEY'] = '123GoniszTy!'


class PriorityType:
    …

class NotificationPriorities:
    …

@app.route('/notification', methods=['GET', 'POST'])
def notification():

    …

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/')
def index():
    return render_template('index.html')
```


RAFAŁ MOBILO24.EU © 2020

Stosowanie formatów bootsrap

Propozycja rozwiązania

```
# form.html - only the modified part

<form id="notification_form" method="POST" action="{{ url_for('notification') }}">
    <div class="container px-4">
        <div class="row mb-3">
            <div class="col-12"><h2>Notify shift manager:</h2></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="room_number" class="form-label">Room
number</label></div>
            <div class="col-6"><input type="text" name="room_number" id="room_number"
class="form-control"></div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="guest_name" class="form-label">Guest
name</label></div>
            <div class="col-6"><input type="text" name="guest_name" id="guest_name" class="form-
control"></div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="notification_text" class="form-
label">Notification</label></div>
            <div class="col-6"><textarea rows="4" cols="50" name="notification_text" class="form-
control"></textarea></div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="priority" class="form-label">Priority</label></div>
            <div class="col-6">
                <select name="priority" id="priority" class="form-select">
                    {% for priority in list_of_priorities %}
                        <option value="{{ priority.code }}" {{ 'selected' if
priority.selected}}>{{ priority.description }}</option>
                    {% endfor %}
                </select>
            </div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"></div>
            <div class="col-6"><input type="submit" value="Send" class="btn btn-primary"></div>
            <div class="col-3"></div>
        </div>
    </div>

</form>


# notification_confirmation.html


    <div class="container px-4">
        <div class="row mb-3">
            <div class="col-12">
                <h2>
                    {% if priority_type.code=='high' %}
                        Critical notification content
                    {% elif priority_type.code=='medium' %}
                        Important notification content
                    {% else %}
                        Notification content
                    {% endif %}
                </h2>
            </div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="room_number" class="form-label">Room
number</label></div>
            <div class="col-6"><input type="text" name="room_number" id="room_number"
class="form-control"  value="{{ room_number }}" readonly></div>
            <div class="col-3"></div>
```

```
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="guest_name" class="form-label">Guest
name</label></div>
            <div class="col-6"><input type="text" name="guest_name" id="guest_name" class="form-
control" value="{{ guest_name }}" readonly></div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="notification_text" class="form-
label">Notification</label></div>
            <div class="col-6">
                <textarea rows="4" cols="50" name="notification_text" class="form-control">
                    {{ notification_text }}
                </textarea>
            </div>
            <div class="col-3"></div>
        </div>
    </div>
```


RAFAŁ MOBILO24.EU © 2020

```
def read_csv_data():

    full_file_path = os.path.join(app.static_folder, 'trips.txt')
    fieldnames = ['trip_name', 'email', 'description', 'completness', 'contact_ok']

    entries = []
    with open(full_file_path, mode='r', encoding="utf-8") as f:
        csv_reader = csv.DictReader(f, fieldnames=fieldnames)
        line_count = 0
        for row in csv_reader:
            if line_count == 0:
                line_count += 1
            else:
                entries.append(row)
            line_count += 1
    return entries

def append_csv_data(data):

    full_file_path = os.path.join(app.static_folder, 'trips.txt')
    fieldnames = ['trip_name', 'email', 'description', 'completness', 'contact_ok']

    if not os.path.exists(full_file_path):
        with open(full_file_path, 'w+', newline='',encoding="utf-8") as f:
                writer = csv.DictWriter(f, fieldnames=fieldnames)
                writer.writeheader()

    with open(full_file_path, 'a', newline='',encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writerow(data)
```

Propozycja rozwiązania (fragmenty)

Przyjmowanie parametrów formularza:

```
    if request.method == 'GET':
        return render_template('new_trip.html',
            trip_name=trip_name, email=email, description=description, completness=completness, c
onctact_ok=contact_ok)
    else:
        trip_name = '' if 'trip_name' not in request.form else request.form['trip_name']
        email = '' if 'email' not in request.form else request.form['email']
        description = '' if 'description' not in request.form else request.form['description']
        completness = False if 'completness' not in request.form else request.form['completness']
=='yes'
        contact_ok = False if 'contact_ok' not in request.form else True
```

Kod formularza przyjmującego pomysły:

```
<form id="trip_form" method="POST" action="{{ url_for('new_trip') }}"

<div class="container">
    <div class="row mb-3">
      <div class="col-1">
      </div>
      <div class="col-10">
        <div class="form-group row mb-3">
            <label for="trip_name" class="col-sm-4 col-form-label">Trip name</label>
            <div class="col-sm-8">
              <input type="text" class="form-
control" id="trip_name" name="trip_name" value="{{trip_name}}">
            </div>
          </div>
          <div class="form-group row mb-3">
            <label for="email" class="col-sm-4 col-form-label">Your email</label>
            <div class="col-sm-8">
              <input type="email" class="form-control" id="email" name="email" value="{{email}}">
            </div>
          </div>
          <div class="form-group row mb-3">
            <label for="description" class="col-sm-4 col-form-label">Short description</label>
            <div class="col-sm-8">
              <textarea rows="4" cols="50" class="form-
control" id="description" name="description">{{description}}</textarea><br>
            </div>
          </div>
          <fieldset class="form-group mb-3">
            <div class="row">
              <legend class="col-form-label col-sm-4 pt-0">Completness</legend>
              <div class="col-sm-8">
                <div class="form-check">
                  <input class="form-check-
input" type="radio" name="completness" id="completness_1" value="yes" {{'checked' if completness
else ''}}>
                  <label class="form-check-label" for="completness_1">
                    Yes - the idea is complete including price proposal
                  </label>
                </div>
                <div class="form-check">
                  <input class="form-check-
input" type="radio" name="completness" id="completness_2" value="no" {{'checked' if not completne
ss}}>
                  <label class="form-check-label" for="completness_2">
                    No - this is just pure idea
                  </label>
                </div>
              </div>
            </div>
          </fieldset>
          <div class="form-group row mb-3">
            <div class="col-sm-4">May we contact you for details?</div>
            <div class="col-sm-8">
              <div class="form-check">
                <input class="form-check-
input" type="checkbox" id="contact_ok" name="contact_ok" value="yes" {{'checked' if contact_ok}}>
                <label class="form-check-label" for="contact_ok">
                  Yes, I agree
                </label>
              </div>
            </div>
          </div>
          <div class="form-group row mb-3">
            <div class="col-sm-12">
              <button type="submit" class="btn btn-success btn-block">Send proposal</button>
            </div>
          </div>
        </div>
        <div class="col-1">
        </div>
      </div>
    </div>
</form>
```


RAFAŁ MOBILO24.EU © 2020

Propozycja rozwiązania

```
sqlite3 notification.db

sqlite> create table notifications(id integer primary key autoincrement, room_number
varchar(10), guest_name varchar(30), notification text, priority varchar(20));

sqlite> insert into notifications(room_number, guest_name, notification, priority)
values('10A', 'Frederico Romane', 'The window cannnot be open', 'MEDIUM');

sqlite>.help

sqlite> .header on

sqlite> select * from notifications;
id|room_number|guest_name|notification|priority
1|10A|Frederico Romane|The window cannnot be open|MEDIUM

.quit
```

Zapis danych z rekordu do bazy danych

Propozycja rozwiązania

```
from flask import Flask, url_for, request, redirect, render_template, flash, g
from datetime import datetime
import sqlite3
import os

app = Flask(__name__)
app.config['SECRET_KEY'] = '123GoniszTy!'

app_info = { ‘db_file' : r"D:\FLASK\hotel\data\notification.db" }




def get_db():
    if not hasattr(g, 'sqlite_db'):
        conn = sqlite3.connect(app_info['db_file'])
        conn.row_factory = sqlite3.Row
        g.sqlite_db = conn
    return g.sqlite_db

@app.teardown_appcontext
def close_db(error):
    if hasattr(g, 'sqlite_db'):
        g.sqlite_db.close()




@app.route('/notification', methods=['GET', 'POST'])
def notification():

    notification_priorities = NotificationPriorities()
    notification_priorities.load_priorities()

    if request.method == 'GET':
        return render_template('notification.html',
                               list_of_priorities=notification_priorities.list_of_priorities)
    else:
        room_number = request.form['room_number'] if 'room_number' in request.form else ''
        guest_name = request.form['guest_name'] if 'guest_name' in request.form else ''
        notification_text =  request.form['notification_text'] if 'notification_text' in
request.form else ''
        priority = request.form['priority'] if 'priority' in request.form else 'normal'

        priority_type = notification_priorities.get_priority_by_code(priority)
        print('found', priority_type.code)

        flash('Notification has been sent')

        the_hour = datetime.now().hour
        raise_priority = (the_hour >= 20 or the_hour < 10) and priority == 'medium'

        if raise_priority:
            priority = 'high'
            flash('Rising priority from medium to high')

        db = get_db()
        sql_command = 'insert into notifications(room_number, guest_name, notification, priority)
values(?, ?, ?, ?)'
        db.execute(sql_command, [room_number, guest_name, notification_text, priority])
        db.commit()

        return render_template('notification_content.html',
                room_number=room_number, guest_name=guest_name,
                notification_text=notification_text, priority_type=priority_type)
```


RAFAŁ MOBILO24.EU © 2020

Pobieranie rekordów z bazy danych

Propozycja rozwiązania

```
# app.py - only the new notification function (add active_menu param to each render template):

@app.route('/notifications')
def notifications():
    db = get_db()
    sql_command = 'select id, room_number, guest_name, notification, priority from
notifications;'
    cur = db.execute(sql_command)
    notifications = cur.fetchall()

    return render_template('notifications.html', active_menu='notifications',
notifications=notifications)

# notifications.html

{% extends "base.html" %}

{% block title %}
    Current notifications
{% endblock %}

{% block content %}
<div class="container">
    <table class="table">
        <thead>
          <tr>
            <th scope="col">#</th>
            <th scope="col">Room Number</th>
            <th scope="col">Guest Name</th>
            <th scope="col">Notification</th>
            <th scope="col">Priority</th>
            <th scope="col">Operations</th>
          </tr>
        </thead>
        <tbody>
          {% for n in notifications %}
          <tr>
            <th scope="row">{{ n.id }}</th>
            <td>{{ n.room_number }}</td>
            <td>{{ n.guest_name }}</td>
            <td>{{ n.notification }}</td>
            <td>{{ n.priority }}</td>
            <td>
                <a href="#" class="btn btn-primary btn-sm" role="button">Actions...</a>
                <a href="#" class="btn btn-success btn-sm" role="button">Edit...</a>
                <a href="#" class="btn btn-danger btn-sm" role="button">Delete...</a>
            </td>
          </tr>
          {%endfor%}
        </tbody>
      </table>
</div>
{% endblock %}
```

```
# menu.html

<ul class="nav nav-pills">
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='index' }}" aria-current="page" href="{{
url_for('index') }}">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='notifications' }}" href="{{
url_for('notifications') }}">Notifications</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='notification' }}" href="{{
url_for('notification') }}">New notification</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='about' }}" href="{{ url_for('about') }}"
tabindex="-1">About this app</a>
    </li>
  </ul>
```


RAFAŁ MOBILO24.EU © 2020

Kasowanie rekordu z pytaniem o potwierdzenie

Propozycja rozwiązania

```
# file notifications.html - only fragment with buttons:

<a href="#" class="btn btn-primary btn-sm" role="button">Actions...</a>
<a href="#" class="btn btn-success btn-sm" role="button">Edit...</a>

<a type="button" class="btn btn-danger btn-sm delete-confirm"
   data-bs-toggle="modal" data-bs-target="#confirmDeleteModal"
   data-desc="{{ 'Delete notification for {} {}?'.format(n.room_number, n.guest_name)
}}"
   data-url="{{ url_for('delete_notification', notification_id=n.id) }}">
  Delete
</a>


# file app.py - only method deleting a row

@app.route('/delete_notification/<int:notification_id>')
def delete_notification(notification_id):

    db = get_db()
    sql_statement = 'delete from notifications where id = ?;'
    db.execute(sql_statement, [notification_id])
    db.commit()

    return redirect(url_for('notifications'))
```

Modyfikacja danych

Propozycja rozwiązania

```
# app.py - only the edit_notification function:

@app.route('/edit_notification/<int:notification_id>', methods=['GET', 'POST'])
def edit_notification(notification_id):
    db = get_db()
    notification_priorities = NotificationPriorities()
    notification_priorities.load_priorities()
    if request.method == 'GET':
        sql_statement = 'select id, room_number, guest_name, notification, priority from
notifications where id = ?'
        cur = db.execute(sql_statement, [notification_id])
        notif_obj = cur.fetchone()
        if notif_obj == None:
            flash('No such notification...')
            return redirect('notifications')
        else:
            return render_template('edit_notification.html', active_menu='notifications',
notif_obj=notif_obj,
                           list_of_priorities=notification_priorities.list_of_priorities)
    else:
        room_number = request.form['room_number'] if 'room_number' in request.form else ''
        guest_name = request.form['guest_name'] if 'guest_name' in request.form else ''
        notification_text =  request.form['notification_text'] if 'notification_text' in
request.form else ''
        priority = request.form['priority'] if 'priority' in request.form else 'normal'
        priority_type = notification_priorities.get_priority_by_code(priority)

        sql_command = '''update notifications set room_number = ?, guest_name = ?,
                            notification = ?, priority = ? where id = ? '''
        db.execute(sql_command, [room_number, guest_name, notification_text, priority,
notification_id])
        db.commit()
        flash('Notification has been updated')
        return redirect(url_for('notifications'))

# notifications.html - how to start edit action:

<a href="{{ url_for('edit_notification', notification_id=n.id) }}" class="btn btn-success btn-sm"
role="button">Edit...</a>

# edit_notification.html

{% extends "base.html" %}
{% block title %} Hotel notifications {% endblock %}
{%block content %}
<form id="notification_form" method="POST" action="{{ url_for('edit_notification',
notification_id=notif_obj.id) }}">
    <div class="container px-4">
        <div class="row mb-3">
            <div class="col-12"><h2>Edit notification details:</h2></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="room_number" class="form-label">Room
number</label></div>
            <div class="col-6"><input type="text" name="room_number" id="room_number"
class="form-control"
                value="{{ notif_obj.room_number }}"></div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="guest_name" class="form-label">Guest
name</label></div>
            <div class="col-6"><input type="text" name="guest_name" id="guest_name" class="form-
control"
                value="{{ notif_obj.guest_name }}"></div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="notification_text" class="form-
label">Notification</label></div>
            <div class="col-6">
                <textarea rows="4" cols="50" name="notification_text" class="form-control">{{
notif_obj.notification }}</textarea>
```


RAFAŁ MOBILO24.EU © 2020

```
        </div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"><label for="priority" class="form-label">Priority</label></div>
            <div class="col-6">
                <select name="priority" id="priority" class="form-select">
                    {% for priority in list_of_priorities %}
                        <option value="{{ priority.code }}" {{ 'selected' if
priority.code==notif_obj.priority }}>{{ priority.description }}</option>
                    {% endfor %}
                </select>
            </div>
            <div class="col-3"></div>
        </div>
        <div class="row mb-3">
            <div class="col-3"></div>
            <div class="col-6"><input type="submit" value="Send" class="btn btn-primary"></div>
            <div class="col-3"></div>
        </div>
    </div>

</form>
{% endblock %}
```

Użytkownicy aplikacji - przygotowanie

Propozycja rozwiązania

```
# SQL to create a table

CREATE TABLE users(
    id integer primary key autoincrement,
    name varchar(100) not null unique,
    email varchar(100) not null unique,
    password text,
    is_active boolean not null default 0,
    is_admin boolean  not null default 0
);


# app.py - import statements


import random
import string
import hashlib
import binascii


# app.py - helper UserPass class


class UserPass:

    def __init__(self, user='', password=''):
        self.user = user
        self.password = password

    def hash_password(self):
        """Hash a password for storing."""
        # the value generated using os.urandom(60)
        os_urandom_static =
b"ID_\x12p:\x8d\xe7&\xcb\xf0=H1\xc1\x16\xac\xe5BX\xd7\xd6j\xe3i\x11\xbe\xaa\x05\xccc\xc2\xe8K\xcf
\xf1\xac\x9bFy(\xfbn.`\xe9\xcd\xdd'\xdf~vm\xae\xf2\x93WD\x04"
        salt = hashlib.sha256(os_urandom_static).hexdigest().encode('ascii')
        pwdhash = hashlib.pbkdf2_hmac('sha512', self.password.encode('utf-8'), salt, 100000)
        pwdhash = binascii.hexlify(pwdhash)
        return (salt + pwdhash).decode('ascii')

    def verify_password(self, stored_password, provided_password):
        """Verify a stored password against one provided by user"""
        salt = stored_password[:64]
        stored_password = stored_password[64:]
        pwdhash = hashlib.pbkdf2_hmac('sha512', provided_password.encode('utf-8'),
salt.encode('ascii'),  100000)
        pwdhash = binascii.hexlify(pwdhash).decode('ascii')
        return pwdhash == stored_password

    def get_random_user_pasword(self):
        random_user = ''.join(random.choice(string.ascii_lowercase)for i in range(3))
        self.user = random_user

        password_characters = string.ascii_letters #+ string.digits + string.punctuation
        random_password = ''.join(random.choice(password_characters)for i in range(3))
        self.password = random_password


# app.py - init route and function


@app.route('/init_app')
def init_app():

    # check if there are users defined (at least one active admin required)
    db = get_db()
    sql_statement = 'select count(*) as cnt from users where is_active and is_admin;'
    cur = db.execute(sql_statement)
    active_admins = cur.fetchone()
```


RAFAŁ MOBILO24.EU © 2020

```
    if active_admins!=None and active_admins['cnt']>0:
        flash('Application is already set-up. Nothing to do')
        return redirect(url_for('index'))

    # if not - create/update admin account with a new password and admin privileges, display
random username
    user_pass = UserPass()
    user_pass.get_random_user_pasword()
    sql_statement = '''insert into users(name, email, password, is_active, is_admin)
                       values(?,?,?,True, True);'''
    db.execute(sql_statement, [user_pass.user, 'noone@nowhere.no', user_pass.hash_password()])
    db.commit()
    flash('User {} with password {} has been created'.format(user_pass.user, user_pass.password))
    return redirect(url_for('index'))
```

Sesja w akcji – logowanie i wylogowanie użytkownika

Propozycja rozwiązania

```
# app.py

from flask import session

class UserPass:
    # … - the code as in the previous version

    def login_user(self):

        db = get_db()
        sql_statement = 'select id, name, email, password, is_active, is_admin from users where
name=?'
        cur = db.execute(sql_statement, [self.user])
        user_record = cur.fetchone()

        if user_record != None and self.verify_password(user_record['password'], self.password):
            return user_record
        else:
            self.user = None
            self.password = None
            return None


@app.route('/login', methods=['GET','POST'])
def login():

    if request.method == 'GET':
        return render_template('login.html', active_menu='login')
    else:
        user_name = '' if 'user_name' not in request.form else request.form['user_name']
        user_pass = '' if 'user_pass' not in request.form else request.form['user_pass']

        login = UserPass(user_name, user_pass)
        login_record = login.login_user()

        if login_record != None:
            session['user'] = user_name
            flash('Logon succesfull, welcome {}'.format(user_name))
            return redirect(url_for('index'))
        else:
            flash('Logon failed, try again')
            return render_template('login.html')

@app.route('/logout')
def logout():

    if 'user' in session:
        session.pop('user', None)
        flash('You are logged out')
    return redirect(url_for('login'))

# login.html
{% extends "base.html" %}
{% block title %}Logon{% endblock %}

{% block content %}
<div class="container">
    <form method="POST" action={{url_for('login')}}>
        <div class="row mt-3">
            <div class="col-6 h1">Login</div>
        </div>

        <div class="row mt-3">
            <div class="col-2 col-form-label"><label for="user_name">User name</label></div>
            <div class="col-4">
                <input type="text" id="user_name" name="user_name" class="form-control">
            </div>
        </div>

        <div class="row mt-3">
            <div class="col-2 col-form-label"><label for="user_pas">Password</label></div>
            <div class="col-4">
                <input type="password" id="user_pass" name="user_pass" class="form-control">
```


RAFAŁ MOBILO24.EU © 2020

```
            </div>
        </div>

        <div class="row mt-3">
            <div class="col-2"></div>
            <div class="col-4"><input type="submit" value="Login" class="btn btn-primary"></div>
        </div>
    </form>
</div>
{% endblock %}

# menu.html

    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='login' }}" href="{{ url_for('login') }}"
tabindex="-1">Login</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='logout' }}" href="{{ url_for('logout') }}"
tabindex="-1">
        Logout {{ session['user'] if 'user' in session }}</a>
    </li>
```

Dodawanie użytkowników – zadbaj o wygodę użytkownika

Propozycja rozwiązania

```
# to add to app.py

@app.route('/users')
def users():
    return 'not implemented'

@app.route('/user_status_change/<action>/<user_name>')
def user_status_change(action, user_name):
    return 'not implemented'

@app.route('/edit_user/<user_name>', methods=['GET', 'POST'])
def edit_user(user_name):
    return 'not implemented'


@app.route('/user_delete/<user_name>')
def delete_user(user_name):
    return 'not implemented'

@app.route('/new_user', methods=['GET', 'POST'])
def new_user():

    if not 'user' in session:
        return redirect(url_for('login'))
    login = session['user']

    db = get_db()
    message = None
    user = {}

    if request.method =='GET':
        return render_template('new_user.html', active_menu='users', user=user)
    else:
        user['user_name'] = '' if not 'user_name' in request.form else request.form['user_name']
        user['email'] = '' if not 'email' in request.form else request.form['email']
        user['user_pass'] = '' if not 'user_pass' in request.form else request.form['user_pass']

        cursor = db.execute('select count(*) as cnt from users where name = ?',
[user['user_name']])
        record = cursor.fetchone()
        is_user_name_unique = (record['cnt'] == 0)

        cursor = db.execute('select count(*) as cnt from users where email = ?', [user['email']])
        record = cursor.fetchone()
        is_user_email_unique = (record['cnt'] == 0)

        if user['user_name'] == '':
            message = 'Name cannot be empty'
        elif user['email'] == '':
            message = 'email cannot be empty'
        elif user['user_pass'] == '':
            message = 'Password cannot be empty'
        elif not is_user_name_unique:
            message = 'User with the name {} already exists'.format(user['user_name'])
        elif not is_user_email_unique:
            message = 'User with the email {} alresdy exists'.format(user['email'])

        if not message:
            user_pass = UserPass(user['user_name'], user['user_pass'])
            password_hash = user_pass.hash_password()
            sql_statement = '''insert into users(name, email, password, is_active, is_admin)
                        values(?,?,?, True, False);'''
            db.execute(sql_statement, [user['user_name'], user['email'], password_hash])
            db.commit()
            flash('User {} created'.format(user['user_name']))
            return redirect(url_for('users'))
        else:
            flash('Correct error: {}'.format(message))
            return render_template('new_user.html', active_menu='users', user=user)

# to add to menu.html

          <li class="nav-item dropdown">
```


RAFAŁ MOBILO24.EU © 2020

```
            <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button"
                data-bs-toggle="dropdown" aria-expanded="false">
              Users
            </a>
            <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
              <li><a class="dropdown-item" href="{{ url_for('users') }}">Users</a></li>
              <li><a class="dropdown-item" href="{{ url_for('new_user') }}">New user</a></li>
            </ul>
          </li>

# new_user.html

{% extends "base.html" %}
{% block content %}
<div class="container">
    <div class="row mb-3">
        <div class="col-5 h1">New user</div>
    </div>
    <form id="user_form" action="{{ url_for('new_user') }}" method="POST">

        <div class="row mb-3">
            <div class="col-1 col-form-label"><label for="user_name">User name</label></div>
            <div class="col-4">
                <input type="text" id="user_name" name="user_name" value="{{ user.user_name }}"
class="form-control"><br>
            </div>
        </div>
        <div class="row mb-3">
            <div class="col-1 col-form-label"><label for="email">Email</label></div>
            <div class="col-4">
                <input type="text" id="email" name="email" value="{{ user.email }}" class="form-
control"><br>
            </div>
        </div>
        <div class="row  mb-3">
            <div class="col-1 col-form-label"><label for="user_pass">Password</label></div>
            <div class="col-4">
                <input type="password" id="user_pass" name="user_pass" class="form-control"><br>
            </div>
        </div>
        <div class="row mb-3">
            <div class="col-1"></div>
            <div class="col-2"><input type="submit" value="Send" class="btn btn-primary"></div>
        </div>
    </form>
</div>
{% endblock %}
```

Lista użytkowników

Propozycja rozwiązania

```
# users.html - the most important points only

{% block content %}
<!-- Modal -->
<div class="modal fade" id="confirmDeleteModal" tabindex="-1" aria-labelledby="exampleModalLabel"
aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="exampleModalLabel">This entry will be deleted:</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-
label="Close"></button>
      </div>
      <div class="modal-body" id="idDeleteModalBody">
        ...
      </div>
      <div class="modal-footer">
        <form action="" method="GET" id="confirmDeleteForm">
          <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
          <button type="submit" class="btn btn-danger">Delete</button>
        </form>
      </div>
    </div>
  </div>
</div>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script>
  $(document).ready(function () {
      // For A Delete Record Popup
      // This function is applied to all elements with class ending  with ".delete-confirm"
      $('.delete-confirm').click(function () {
          // get attributes of the found element
          var desc = $(this).attr('data-desc');
          var url = $(this).attr('data-url');
          // the #... designates id of an element - change the text in the modal window
          $('#idDeleteModalBody').text(desc);
          $("#confirmDeleteForm").attr("action", url);
      });
  });
</script>

<div class="container">
    <table class="table">
        <thead>
          <tr>
            <th scope="col">#</th>
            <th scope="col">User name</th>
            <th scope="col">Email</th>
            <th scope="col">Is active</th>
            <th scope="col">Is admin</th>
            <th scope="col">Actions</th>
          </tr>
        </thead>
        <tbody>
          {% for user in users %}
          <tr>
            <th scope="row">{{ user.id }}</th>
            <td>{{ user.name }}</td>
            <td>{{ user.email }}</td>
            <td>       </td>
            <td>       </td>
            <td>
                <a href="{{ url_for('edit_user', user_name=user.name) }}"
                   class="btn btn-success btn-sm" role="button">Edit...</a>
                <a type="button" class="btn btn-danger btn-sm delete-confirm"
                   data-bs-toggle="modal" data-bs-target="#confirmDeleteModal"
                   data-desc="{{ 'Delete user {}?'.format(user.name) }}"
                   data-url="{{ url_for('delete_user', user_name=user.name) }}">
                  Delete
                </a>
            </td>
          </tr>
```


RAFAŁ MOBILO24.EU © 2020

```
          {%endfor%}
        </tbody>
      </table>
</div>
{% endblock %}


# app.py - only the most important functions:

@app.route('/users')
def users():

    db = get_db()
    sql_command = 'select id, name, email, is_admin, is_active from users;'
    cur = db.execute(sql_command)
    users = cur.fetchall()

    return render_template('users.html', active_menu='users', users=users)



@app.route('/user_delete/<user_name>')
def delete_user(user_name):

    if not 'user' in session:
        return redirect(url_for('login'))
    login = session['user']

    db = get_db()
    sql_statement = "delete from users where name = ? and name <> ?"
    db.execute(sql_statement, [user_name, login])
    db.commit()

    return redirect(url_for('users'))
```

Edycja użytkownika

Propozycja rozwiązania

```
# app.py - only modified function

@app.route('/edit_user/<user_name>', methods=['GET', 'POST'])
def edit_user(user_name):

    db = get_db()
    cur = db.execute('select name, email from users where name = ?', [user_name])
    user = cur.fetchone()
    message = None

    if user == None:
        flash('No such user')
        return redirect(url_for('users'))

    if request.method == 'GET':
        return render_template('edit_user.html', active_menu='users', user=user)
    else:
        new_email = '' if 'email' not in request.form else request.form["email"]
        new_password = '' if 'user_pass' not in request.form else request.form['user_pass']

        if new_email != user['email']:
            sql_statement = "update users set email = ? where name = ?"
            db.execute(sql_statement, [new_email, user_name])
            db.commit()
            flash('Email was changed')

        if new_password != '':
            user_pass = UserPass(user_name, new_password)
            sql_statement = "update users set password = ?  where name = ?"
            db.execute(sql_statement, [user_pass.hash_password(), user_name])
            db.commit()
            flash('Password was changed')

        return redirect(url_for('users'))


# edit_user.html



{% extends "base.html" %}

{% block title %}
    Edit user
{% endblock %}

{% block content %}
<div class="container">
    <div class="row mb-3">
        <div class="col-5 h1">Edit user {{ user.name }}</div>
    </div>
    <form id="user_form" action="{{ url_for('edit_user', user_name=user.name) }}" method="POST">

        <div class="row mb-3">
            <div class="col-1 col-form-label"><label for="email">Email</label></div>
            <div class="col-4">
                <input type="text" id="email" name="email" value="{{ user.email }}" class="form-
control"><br>
            </div>
        </div>

        <div class="row  mb-3">
            <div class="col-1 col-form-label"><label for="user_pass">Password</label></div>
            <div class="col-4">
                <input type="password" id="user_pass" name="user_pass" class="form-control"><br>
            </div>
        </div>

        <div class="row mb-3">
            <div class="col-1"></div>
            <div class="col-2"><input type="submit" value="Send" class="btn btn-primary"></div>
        </div>
```


RAFAŁ MOBILO24.EU © 2020

```
    </form>
</div>
{% endblock %}
```

Edycja uprawnień

Propozycja rozwiązania

```
# app.py - only one function:


@app.route('/user_status_change/<action>/<user_name>')
def user_status_change(action, user_name):

    if not 'user' in session:
        return redirect(url_for('login'))
    login = session['user']

    db = get_db()

    if action == 'active':
        db.execute("""update users set is_active = (is_active + 1) % 2
                      where name = ? and name <> ?""",
                      [user_name, login])
        db.commit()
    elif action == 'admin':
        db.execute("""update users set is_admin = (is_admin + 1) % 2
                      where name = ? and name <> ?""",
                      [user_name, login])
        db.commit()

    return redirect(url_for('users'))
```

```
# users.html - missing part of the table


<td>
    <a href="{{ url_for('user_status_change', action='active', user_name=user.name)
}}">
        {% if user.is_active %}
           &check;
        {% else %}
           &#x25a2;
        {% endif %}
    </a>
</td>
<td>
    <a href="{{ url_for('user_status_change', action='admin', user_name=user.name)
}}">
        {% if user.is_admin %}
           &check;
        {% else %}
           &#x25a2;
        {% endif %}
    </a>
</td>
```


RAFAŁ MOBILO24.EU © 2020

Propozycja rozwiązania

```
# app.py - properties and method for UserPass class:

class UserPass:

    def __init__(self, user='', password=''):
        self.user = user
        self.password = password
        self.email = ''
        self.is_valid = False
        self.is_admin = False


    def get_user_info(self):
        db = get_db()
        sql_statement = 'select name, email, is_active, is_admin from users where name=?'
        cur = db.execute(sql_statement, [self.user])
        db_user = cur.fetchone()

        if db_user == None:
            self.is_valid = False
            self.is_admin = False
            self.email = ''
        elif db_user['is_active']!=1:
            self.is_valid = False
            self.is_admin = False
            self.email = db_user['email']
        else:
            self.is_valid = True
            self.is_admin = db_user['is_admin']
            self.email = db_user['email']


# app.py - code to add to functions - non admin access:

    login = UserPass(session.get('user'))
    login.get_user_info()
    if not login.is_valid:
        return redirect(url_for('login'))


# app.py - code to add to functions - admin access

    login = UserPass(session.get('user'))
    login.get_user_info()
    if not login.is_valid or not login.is_admin:
        return redirect(url_for('login'))


# menu.html - code to show/hide different menu options (only 2 examples):

    {% if login.is_valid: %}
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='notification' }}" href="{{
url_for('notification') }}">New notification</a>
    </li>
    {% endif %}
    <li class="nav-item">
      <a class="nav-link {{ 'active' if active_menu=='about' }}" href="{{ url_for('about') }}"
tabindex="-1">About this app</a>
    </li>
    {% if login.is_valid and login.is_admin: %}
    <li class="nav-item dropdown">
      <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button"
           data-bs-toggle="dropdown" aria-expanded="false">
        Users
      </a>
      <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
        <li><a class="dropdown-item" href="{{ url_for('users') }}">Users</a></li>
        <li><a class="dropdown-item" href="{{ url_for('new_user') }}">New user</a></li>
      </ul>
    </li>
    {% endif %}
```

Flask SQLAlchemy

Propozycja rozwiązania

```
#app.py

from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config.from_pyfile('config.cfg')
db = SQLAlchemy(app)

class Author(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))
    special = db.Column(db.Boolean)

@app.route('/')
def index():
    db.create_all()

    return '''Hello Flask-SQLAlchemy
    '''

if __name__ == '__main__':
    app.run()



# config.cfg

SQLALCHEMY_DATABASE_URI='sqlite:///c:\\data\\museum.db'
SQLALCHEMY_TRACK_MODIFICATIONS=False
```


RAFAŁ MOBILO24.EU © 2020

Dodawanie I pobieranie danych z bazy danych

Propozycja rozwiązania

```
# app.py - class definition

class Author(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))
    special = db.Column(db.Boolean)

    def __repr__(self):
        return '<id: {}, name: {}, special: {}>'.format(self.id, self.name,
self.special)


# in python session:

>>> from app import app, db, Author
>>> dali = Author(id=1, name='Salvador Dali', special=False)
>>> db.session.add(dali)
>>> picasso = Author(id=2, name='Pablo Picasso', special=False)
>>> db.session.add(picasso)
>>> cezane = Author(id=3, name='Paul Cezane', special=True)
>>> db.session.add(cezane)
>>> db.session.commit()
>>> Author.query.all()
[<id: 1, name: Salvador Dali, special: False>, <id: 2, name: Pablo Picasso, special:
False>, <id: 3, name: Paul Cezane, special: True>]
>>>
```

Filtrowanie danych

Propozycja rozwiązania

```
# in python session:

>>> from app import app, db, Authors

>>> monet = Author(id=4, name='Cloud Monet', special=False)
>>> db.session.add(monet)
>>> warhol = Authors(id=5, name='Andy Warhol')
>>> db.session.add(warhol)
>>> kahlo = Author(id=6, name='Frida Kahlo')
>>> db.session.add(kahlo)
>>> db.session.commit()

>>> Author.query.all()

>>> Author.query.filter(Author.id.in_([1,3,5])).all()

>>> Author.query.filter(Author.special == None).all()

>>> Author.query.filter(Author.special != None).all()

>>> Author.query.filter(Author.name.like('%u%')).all()

>>> Author.query.filter(db.and_(Author.name.like('%w%'), Author.special==None)).all()

>>> Author.query.filter(db.or_(Author.name.like('%w%'), Author.special==None)).all()

>>> Author.query.filter(db.and_(~Author.name.like('%w%'), Author.special==None)).all()

>>> Author.query.filter(db.and_(~Author.name.like('%w%'), Author.special!=None)).all()

>>> Author.query.filter(db.or_(Author.name.like('%w%'), Author.name.like('%u%'))).all()
```


RAFAŁ MOBILO24.EU © 2020

Sortowanie,zliczanie i ograniczanie liczby rekordów

Propozycja rozwiązania

```
Author.query.order_by(Author.name).all()
Author.query.order_by(Author.name.desc()).all()
Author.query.filter(Author.special != None).order_by(Author.name).all()
Author.query.order_by(Author.id).limit(5).all()
Author.query.order_by(Author.id).offset(5).limit(5).all()
Author.query.count()
Author.query.filter(Author.special == None).count()
Author.query.filter(Author.special != None).count()
```

Dodawanie i usuwanie danych

Propozycja rozwiązania

```
Author.query.all()

picasso = Author.query.filter(Author.name == 'Pablo Picasso').first()
picasso.special = True

cezane = Author.query.filter(Author.name == 'Paul Cezane').first()
cezane.special = False

db.session.commit()
Author.query.all()

warhol = Author.query.filter(Author.name == 'Andy Warhol').first()
db.session.delete(warhol)
db.session.commit()

Author.query.all()
```


RAFAŁ MOBILO24.EU © 2020

Propozycja rozwiązania

```
Solution placeholder

from app import db, app, Author, ArtWork
db.create_all()

monet = Author.query.filter(Author.name=='Cloud Monet').first()
nature = ArtWork(id=11, name='The Truth of Nature', author=monet)
sunflowers = ArtWork(id=12, name='Bouquet of Sunflowers', author=monet)

dali = Author.query.filter(Author.name=='Salvador Dali').first()
girafe = ArtWork(id=21, name='Girafe En Feu', author=dali)
anthony = ArtWork(id=22, name='Sant Anthony', author=dali)

db.session.add(nature)
db.session.add(sunflowers)
db.session.add(girafe)
db.session.add(anthony)
db.session.commit()

dali.artwork.all()

ArtWork.query.filter(ArtWork.id == 21).first().author

ArtWork.query.filter(ArtWork.id == 12).first().author = dali
db.session.commit()
```

Porównanie metod pracy z bazą danych

Propozycja rozwiązania

```
# some snippets to reuse in your solution

# old - get user information ------------------------------------------------------------

db = get_db()
sql_statement = 'select id, name, email, password, is_active, is_admin from users where name=?'
cur = db.execute(sql_statement, [self.user])
user_record = cur.fetchone()

# new

user_record = User.query.filter(User.name == self.user).first()

# old - get all users ------------------------------------------------------------------

db = get_db()
sql_command = 'select id, name, email, is_admin, is_active from users;'
cur = db.execute(sql_command)
users = cur.fetchall()

# new

users = User.query.all()

# old - update is_active for a user, but not for a current user ------------------------------

db.execute("""update users set is_active = (is_active + 1) % 2
                      where name = ? and name <> ?""",
                      [user_name, login.user])
# new

user = User.query.filter(User.name == user_name, User.name != login.user).first()
if user:
    user.is_active = (user.is_active + 1) % 2
    db.session.commit()

# old - update email ------------------------------------------------------------------------

sql_statement = "update users set email = ? where name = ?"
db.execute(sql_statement, [new_email, user_name])
db.commit()

# new

user.email = new_email
db.session.commit()
```


RAFAŁ MOBILO24.EU © 2020

```
# old - delete a user ------------------------------------------------------------------------

db = get_db()
sql_statement = "delete from users where name = ? and name <> ?"
db.execute(sql_statement, [user_name, login.user])
db.commit()


# new

user = User.query.filter(User.name==user_name, User.name != login.user).first()
if user:
    db.session.delete(user)
    db.session.commit()

# old - add a new user -----------------------------------------------------------------------

sql_statement = '''insert into users(name, email, password, is_active, is_admin)
                        values(?,?,?, True, False);'''
db.execute(sql_statement, [user['user_name'], user['email'], password_hash])
db.commit()

# new
new_user = User(name=user['user_name'], email=user['email'], password=password_hash,
                          is_active=True, is_admin=False)
db.session.add(new_user)
db.session.commit()
```

Flask-WTF – instalacja i pierwszy formularz

Propozycja rozwiązania

```
# app.py
from flask import Flask, render_template, url_for
from flask_wtf import FlaskForm
from wtforms import StringField, IntegerField, BooleanField, SelectField


app = Flask(__name__)
app.config['SECRET_KEY'] = 'ACompl1cat3dText.'


class TrainInfo(FlaskForm):

    train_number = StringField('Train number')
    is_delayed = BooleanField('Is delayed')
    delay_minutes = IntegerField('Delay in minutes')
    delay_reason = SelectField('Delay reason', choices=['None', 'Weather', 'Failure',
'Other'])



@app.route('/', methods=['POST', 'GET'])
def index():

    form = TrainInfo()
    if form.validate_on_submit():

        return f'''<H1>Hello</H1>
                    <ul>
                        <li>{form.train_number.label}: {form.train_number.data}</li>
                        <li>{form.is_delayed.label}: {form.is_delayed.data}</li>
                        <li>{form.delay_minutes.label}: {form.delay_minutes.data}</li>
                        <li>{form.delay_reason.label}: {form.delay_reason.data}</li>
                    </ul>'''

    return render_template('index.html', form=form)


if __name__ == '__main__':
    app.run()



# index.html


<form method="POST" action="{{ url_for('index') }}">

    {{ form.csrf_token }}
    {{ form.train_number.label }}   {{ form.train_number }} <br/>
    {{ form.is_delayed.label }}     {{ form.is_delayed }} <br/>
    {{ form.delay_minutes.label }}  {{ form.delay_minutes }} <br/>
    {{ form.delay_reason.label }}   {{ form.delay_reason }} <br/>

    <input type="submit" value="GO!">
</form>
```


RAFAŁ MOBILO24.EU © 2020

Sprawdzanie poprawności danych (validators)

Propozycja rozwiązania

```
# app.py - only fragments:

from wtforms.validators import DataRequired, NumberRange, ValidationError

…

class TrainInfo(FlaskForm):

    def start_with2letters(form, field):
        if not (len(field.data) > 2 and field.data[0:2].isalpha()):
            raise ValidationError('Train number must start with 2 letters')

    train_number = StringField('Train number',
                       validators=[DataRequired('Enter train number'), start_with2letters])
    is_delayed = BooleanField('Is delayed')
    delay_minutes = IntegerField('Delay in minutes', validators=[DataRequired('Enter delay'),
                                 NumberRange(min=0, message='The delay must be a number >= 0')])
    delay_reason = SelectField('Delay reason', choices=['None', 'Weather', 'Failure', 'Other'])


# macros.html

{% macro show_validation_results(field) %}
<ul>
    {% for error in field.errors %}
        <li>{{ error }}</li>
    {% endfor %}
</ul>
{% endmacro %}


# index.html

{% from "macros.html" import show_validation_results %}

<form method="POST" action="{{ url_for('index') }}">

    {{ form.csrf_token }}
    {{ form.train_number.label }}   {{ form.train_number }} <br/>
    {{ show_validation_results(form.train_number) }}
    {{ form.is_delayed.label }}     {{ form.is_delayed }} <br/>
    {{ form.delay_minutes.label }}  {{ form.delay_minutes }} <br/>
    {{ show_validation_results(form.delay_minutes) }}
    {{ form.delay_reason.label }}   {{ form.delay_reason }} <br/>

    <input type="submit" value="GO!">
</form>
```

Zapisywanie danych z formularzy w obiektach

Propozycja rozwiązania

```
# app.py - only selected fragments:

from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SECRET_KEY'] = 'ACompl1cat3dText.'
app.config['SQLALCHEMY_DATABASE_URI']='sqlite:///d:\\Flask\\trains.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS']=False
db = SQLAlchemy(app)

class TrainDelay(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    train_number = db.Column(db.String(30))
    is_delayed = db.Column(db.Boolean)
    delay_minutes = db.Column(db.Integer)
    delay_reason = db.Column(db.String(100))

    def __repr__(self):
        return f"{self.train_number} - {self.delay_minutes}"

db.create_all()



@app.route('/', methods=['POST', 'GET'])
def index():

    form = TrainInfo()
    if form.validate_on_submit():

        train_delay = TrainDelay()
        form.populate_obj(train_delay)
        db.session.add(train_delay)
        db.session.commit()

        return f'''<H1>Hello</H1>
                    <ul>
                        <li>{form.train_number.label}: {form.train_number.data}</li>
                        <li>{form.is_delayed.label}: {form.is_delayed.data}</li>
                        <li>{form.delay_minutes.label}: {form.delay_minutes.data}</li>
                        <li>{form.delay_reason.label}: {form.delay_reason.data}</li>
                    </ul>
                    Following record was added to the table in database: {train_delay}
                    '''

    return render_template('index.html', form=form)
```


RAFAŁ MOBILO24.EU © 2020

Dziedziczenie z klasy formularza

Propozycja rozwiązania

```
# app.py - only important fragments:

class TrainDelayOnStation(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    train_number = db.Column(db.String(30))
    is_delayed = db.Column(db.Boolean)
    delay_minutes = db.Column(db.Integer)
    delay_reason = db.Column(db.String(100))
    station = db.Column(db.String(50))

    def __repr__(self):
        return f"{self.station} - {self.train_number} - {self.delay_minutes}"

class TrainInfoOnStation(TrainInfo):
    station = StringField('Station', validators=[DataRequired('Enter station name')])

@app.route('/delay_on_station', methods=['POST', 'GET'])
def delay_on_station():

    form = TrainInfoOnStation()
    if form.validate_on_submit():

        train_delay_on_station = TrainDelayOnStation()
        form.populate_obj(train_delay_on_station)
        db.session.add(train_delay_on_station)
        db.session.commit()

        return f'''<H1>Hello</H1>
                    <ul>
                        <li>{form.train_number.label}: {form.train_number.data}</li>
                        <li>{form.is_delayed.label}: {form.is_delayed.data}</li>
                        <li>{form.delay_minutes.label}: {form.delay_minutes.data}</li>
                        <li>{form.delay_reason.label}: {form.delay_reason.data}</li>
                        <li>{form.station.label}: {form.station.data}</li>
                    </ul>
                    Following record was added to the table in database: {train_delay_on_station}
                    '''

    return render_template('index.html', form=form)


# index.html


{% from "macros.html" import show_validation_results %}

<form method="POST" }}">

    {{ form.csrf_token }}
    {% if form.station %}
        {{ form.station.label }}   {{ form.station }} <br/>
        {{ show_validation_results(form.station) }}
    {% endif %}
    {{ form.train_number.label }}   {{ form.train_number }} <br/>
    {{ show_validation_results(form.train_number) }}
    {{ form.is_delayed.label }}     {{ form.is_delayed }} <br/>
    {{ form.delay_minutes.label }}  {{ form.delay_minutes }} <br/>
    {{ show_validation_results(form.delay_minutes) }}
    {{ form.delay_reason.label }}   {{ form.delay_reason }} <br/>

    <input type="submit" value="GO!">
</form>
```

```
    {% for option in form.delay_reason %}
        <tr>
            <td>{{ option }}</td>
            <td>{{ option.label }}</td>
        </tr>
    {% endfor %}
```

Propozycja rozwiązania

```
# app.py - only changed fragments:

from wtforms import StringField, IntegerField, BooleanField, SelectField, RadioField


class TrainInfo(FlaskForm):

    def start_with2letters(form, field):
        if not (len(field.data) > 2 and field.data[0:2].isalpha()):
            raise ValidationError('Train number must start with 2 letters')

    train_number = StringField('Train number',
                       validators=[DataRequired('Enter train number'), start_with2letters])
    is_delayed = BooleanField('Is delayed')
    delay_minutes = IntegerField('Delay in minutes',
                        validators=[DataRequired('Enter delay'),
                             NumberRange(min=0, message='The delay must be a number >= 0')])
    #delay_reason = SelectField('Delay reason', choices=['None', 'Weather', 'Failure', 'Other'])
    delay_reason = RadioField('Delay reason', choices=['None', 'Weather', 'Failure', 'Other'],
                              default='None')




# index.html - full content

{% from "macros.html" import show_validation_results %}

<form method="POST" }}">

    {{ form.csrf_token }}
    {% if form.station %}
        {{ form.station.label }}   {{ form.station }} <br/>
        {{ show_validation_results(form.station) }}
    {% endif %}
    {{ form.train_number.label }}   {{ form.train_number }} <br/>
    {{ show_validation_results(form.train_number) }}
    {{ form.is_delayed.label }}     {{ form.is_delayed }} <br/>
    {{ form.delay_minutes.label }}  {{ form.delay_minutes }} <br/>
    {{ show_validation_results(form.delay_minutes) }}
    <!--{{ form.delay_reason.label }}   {{ form.delay_reason }} <br/> -->
    {% for option in form.delay_reason %}
        <tr>
            <td>{{ option }}</td>
            <td>{{ option.label }}</td>
        </tr>
    {% endfor %}</br>

    <input type="submit" value="GO!">
</form>
```


RAFAŁ MOBILO24.EU © 2020

Propozycja rozwiązania

```
#app.py - only fragments:


class TrainDelayOnStationDay(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    train_number = db.Column(db.String(30))
    is_delayed = db.Column(db.Boolean)
    delay_minutes = db.Column(db.Integer)
    delay_reason = db.Column(db.String(100))
    station = db.Column(db.String(50))
    day = db.Column(db.Date)

    def __repr__(self):
        return f"{self.station} - {self.train_number} - {self.delay_minutes}"


class TrainInfo(FlaskForm):

    def start_with2letters(form, field):
        if not (len(field.data) > 2 and field.data[0:2].isalpha()):
            raise ValidationError('Train number must start with 2 letters')

    train_number = StringField('Train number',
                       validators=[DataRequired('Enter train number'), start_with2letters])
    is_delayed = BooleanField('Is delayed')
    delay_minutes = IntegerField('Delay in minutes', validators=[DataRequired('Enter delay'),
                                 NumberRange(min=0, message='The delay must be a number >= 0')])
    delay_reason = RadioField('Delay reason', choices=['None', 'Weather', 'Failure', 'Other'],
                              default='None')

class TrainInfoOnStation(TrainInfo):
    station = StringField('Station', validators=[DataRequired('Enter station name')])
    day = DateField('Day', validators=[DataRequired('Enter date')], default=date.today())



@app.route('/delay_on_station', methods=['POST', 'GET'])
def delay_on_station():

    form = TrainInfoOnStation()
    if form.validate_on_submit():

        train_delay_on_station_day = TrainDelayOnStationDay()
        form.populate_obj(train_delay_on_station_day)
        db.session.add(train_delay_on_station_day)
        db.session.commit()

        return f'''<H1>Hello</H1>
                    <ul>
                        <li>{form.train_number.label}: {form.train_number.data}</li>
                        <li>{form.is_delayed.label}: {form.is_delayed.data}</li>
                        <li>{form.delay_minutes.label}: {form.delay_minutes.data}</li>
                        <li>{form.delay_reason.label}: {form.delay_reason.data}</li>
                        <li>{form.station.label}: {form.station.data}</li>
                        <li>{form.day.label}: {form.day.data}</li>
                    </ul>
                    Following record was added to the db: {train_delay_on_station_day}
                    '''

    return render_template('index.html', form=form)



#index.html - only added part:


    {% if form.day %}
        {{ form.day.label }}   {{ form.day }} <br/>
        {{ show_validation_results(form.day) }}
    {% endif %}
```

Flask Login – testowa aplikacja

Propozycja rozwiązania

```
# app.py - only changed/added fragments

from flask import Flask, render_template, url_for, request, redirect
from flask_wtf import FlaskForm
from wtforms import BooleanField, StringField, SelectField, RadioField, PasswordField
from wtforms.fields.html5 import IntegerField, DateField
from wtforms.validators import DataRequired, NumberRange, ValidationError
from flask_sqlalchemy import SQLAlchemy
from datetime import date
import hashlib
import binascii

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50), unique=True)
    password = db.Column(db.String(100))
    first_name = db.Column(db.String(50))
    last_name = db.Column(db.String(50))

    def __repr__(self):
        return ('User: {},{}'.format(self.name))

    def get_hashed_password(password):
        """Hash a password for storing."""
        # the value generated using os.urandom(60)
        os_urandom_static =
b"ID_\x12p:\x8d\xe7&\xcb\xf0=H1\xc1\x16\xac\xe5BX\xd7\xd6j\xe3i\x11\xbe\xaa\x05\xccc\xc2\xe8K\xcf
\xf1\xac\x9bFy(\xfbn.`\xe9\xcd\xdd'\xdf~vm\xae\xf2\x93WD\x04"
        salt = hashlib.sha256(os_urandom_static).hexdigest().encode('ascii')
        pwdhash = hashlib.pbkdf2_hmac('sha512', password.encode('utf-8'), salt, 100000)
        pwdhash = binascii.hexlify(pwdhash)
        return (salt + pwdhash).decode('ascii')

    def verify_password(stored_password_hash, provided_password):
        """Verify a stored password against one provided by user"""
        salt = stored_password_hash[:64]
        stored_password = stored_password_hash[64:]
        pwdhash = hashlib.pbkdf2_hmac('sha512', provided_password.encode('utf-8'),
salt.encode('ascii'),  100000)
        pwdhash = binascii.hexlify(pwdhash).decode('ascii')
        return pwdhash == stored_password

class LoginForm(FlaskForm):
    name = StringField('User name')
    password = PasswordField('Password')
    remember = BooleanField('Remember me')


@app.route('/login', methods=['GET','POST'])
def login():
    form = LoginForm()
    return render_template('login.html', form=form)

@app.route('/logout')
def logout():
    return '<h1>You are logged out</h1>'

# login.html - full

<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Bootstrap CSS -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta2/dist/css/bootstrap.min.css"
rel="stylesheet"
          integrity="sha384-BmbxuPwQa2lc/FVzBcNJ7UAyJxM6wuqIj61tLrc4wSX0szH/Ev+nYRRuWlolflfl"
crossorigin="anonymous">
    <title>Logon</title>
  </head>
  <body>
```


RAFAŁ MOBILO24.EU © 2020

```
    {% for message in get_flashed_messages() %}
        <div class="alert alert-warning alert-dismissible fade show" role="alert">
        <strong>{{ message }}</strong>
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
        </div>
    {% endfor %}

    <!--Grid row-->
    <div class="row d-flex justify-content-center">
        <!--Grid column-->
        <div class="col-md-4">

            <form method="POST">
                {{ form.csrf_token }}
                <div class="form-row align-items-center">
                <div class="col-auto my-1">
                    <label class="sr-only" for="inlineFormInputName">{{ form.name.label
}}</label>
                    {{ form.name(class_="form-control") }}
                </div>
                <div class="col-auto my-1">
                    <label class="sr-only" for="inlineFormInputGroupUsername">{{
form.password.label }}</label>
                    {{ form.password(class_="form-control") }}
                </div>
                <div class="col-auto my-1">
                    <div class="form-check">
                    {{ form.remember(class_="form-check-input") }}
                    <label class="form-check-label" for="autoSizingCheck2">
                        {{ form.remember.label }}
                    </label>
                    </div>
                </div>
                <div class="col-auto my-1">
                    <button type="submit" class="btn btn-primary">Login</button>
                </div>
                </div>
            </form>
        </div>
        <!--Grid column-->
    </div>
    <!--Grid row-->

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-
beta2/dist/js/bootstrap.bundle.min.js" integrity="sha384-
b5kHyXgcpbZJO/tY9Ul7kGkf1S0CWuKcCD38l8YkeH8z8QjE0GmW1gYU5S9FOnJ0"
crossorigin="anonymous"></script>

  </body>
</html>
```

Flask-Login instalacja i wykorzystanie

Propozycja rozwiązania

```
#app.py - only fragments

from flask_login import LoginManager, UserMixin, login_user, logout_user, login_required,
current_user

login_manager = LoginManager(app)


class User(db.Model, UserMixin):
  # […]


@login_manager.user_loader
def load_user(id):
    return User.query.filter(User.id == id).first()


@app.route('/init')
def init():
    db.create_all()

    admin = User.query.filter(User.name=='admin').first()
    if admin == None:
        admin = User(id=1, name='admin', password=User.get_hashed_password('PasswOrd'),
                     first_name='King', last_name='Kong')
        db.session.add(admin)
        db.session.commit()

    return '<h1>Initial configuration done!</h1>'


@app.route('/login', methods=['GET','POST'])
def login():
    form = LoginForm()

    if form.validate_on_submit():
        user = User.query.filter(User.name == form.name.data).first()
        if user != None and User.verify_password(user.password, form.password.data):
            login_user(user)

            next = request.args.get('next')
            if next and is_safe_url(next):
                return redirect(next)
            else:
                return '<h1>You are authenticated!</h1>'

    return render_template('login.html', form=form)


@app.route('/logout')
def logout():
    logout_user()
    return '<h1>You are logged out</h1>'


@app.route('/delay_on_station', methods=['POST', 'GET'])
@login_required
def delay_on_station():
    # […]
```


RAFAŁ MOBILO24.EU © 2020

Przekierowanie użytkownika do strony logowania

Propozycja rozwiązania

```
from urllib.parse import urlparse, urljoin



login_manager.login_view = 'login'
login_manager.login_message = 'First, please log in using this form:'




def is_safe_url(target):
    ref_url = urlparse(request.host_url)
    test_url = urlparse(urljoin(request.host_url, target))
    return test_url.scheme in ('http', 'https') and \
           ref_url.netloc == test_url.netloc




@app.route('/login', methods=['GET','POST'])
def login():
    form = LoginForm()

    if form.validate_on_submit():
        user = User.query.filter(User.name == form.name.data).first()
        if user != None and User.verify_password(user.password, form.password.data):
            login_user(user)

            next = request.args.get('next')
            if next and is_safe_url(next):
                return redirect(next)
            else:
                return '<h1>You are authenticated!</h1>'

    return render_template('login.html', form=form)
```

Wymuszenie ponownego logowania

Propozycja rozwiązania

```
# app.py

@app.route('/login', methods=['GET','POST'])
def login():
    form = LoginForm()

    if form.validate_on_submit():
        user = User.query.filter(User.name == form.name.data).first()
        if user != None and User.verify_password(user.password, form.password.data):
            login_user(user, remember=form.remember.data)

            next = request.args.get('next')
            if next and is_safe_url(next):
                return redirect(next)
            else:
                return '<h1>You are authenticated!</h1>'

    return render_template('login.html', form=form)
```


RAFAŁ MOBILO24.EU © 2020

Spróbuj też!


Python
Dla początkujących


Analiza danych
w Python i PANDAS


Python dla średnio
zaawansowanych


Machine Learning
wprowadzenie
perceptron i regresja


Python Flask
Aplikacje webowe


Linux
dla początkujących


Administrator Linux
Instalacja i konfiguracja


PostgreSQL
dla początkujących


Microsoft Azure
AZ-900


Azure Administrator
Storage (AZ-104)

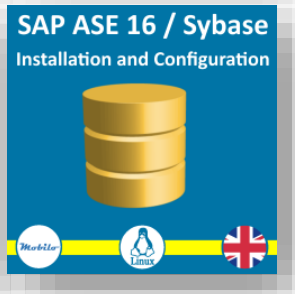

PowerShell
dla administratora Windows


SQL Server #1
Instalacja i narzędzia
Microsoft
SQL Server


SQL Server #2
Zapytania SQL
Microsoft
SQL Server


SQL Server #3
Programowanie
typy zaawansowane, XML
Microsoft
SQL Server


SQL Server #4
Modyfikacja danych
transakcje, obsługa błędów
Microsoft
SQL Server


SAP ASE 16 / Sybase
Installation and Configuration


MS Excel Tips & Tricks


Język R
dla początkujących


Reporting Services
Tworzenie raportów
Microsoft
SQL Server